Spring Boot

Herança

PAULO R T CÂNDIDO

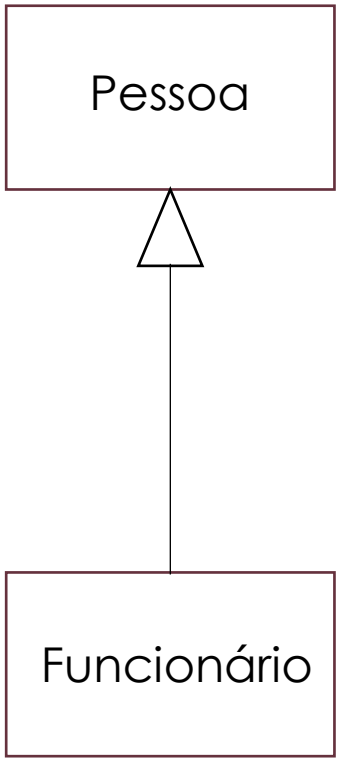
Obter projeto anterior

Obter projeto desenvolvido anteriormente clonando a seguir.

https://github.com/prtcandido/poo.git

Projeto está em exemplopers.zip

Criar Model (Entity) estendendo Pessoa



Funcionário estende Pessoa

Funcionário herda atributos e métodos de Pessoa e adiciona algo específico (cargo e salário).

Generalização e polimorfismo

```
Pessoa p1 = new Pessoa();

Pessoa p2 = new Funcionario();


String s1 = p1.toString(); // chama toString() definido em Pessoa

String s2 = p2.toString(); // chama toString() definido em Funcionario
```

Generalização e polimorfismo

```
public void salvaPessoa(Pessoa p) {...}

// Chama do médodo acima passando um funcionário

Funcionario f = new Funcionario();
        f.setNome("Pedro");
        f.setCargo("Analista");
        f.setSalario(new BigDecimal("8000.00"));
        f.setId(0);

salvaPessoa( f )
```

Pessoa – anotações para persistência/herança

```java
@Entity  // Indica que esta classe é uma entidade JPA, ou seja, será mapeada para uma tabela no banco de dados.
@Table(name = "pessoas") // Especifica o nome da tabela no banco de dados que esta entidade irá mapear.
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(onlyExplicitlyIncluded = true)
@EqualsAndHashCode(onlyExplicitlyIncluded = true)

// Estratégia pode ser: SINGLE_TABLE, JOINED ou TABLE_PER_CLASS
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
// Usado em SINGLE_TABLE para diferenciar subclasses
@DiscriminatorColumn(name = "tipo_pessoa")
// Especifica o valor que será usado na coluna "tipo_pessoa" para identificar esta classe base
@DiscriminatorValue("PESSOA")

public class Pessoa {

    @Id // Indica que o campo "id" é a chave primária da entidade.
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Especifica que o valor do campo "id" será gerado automaticamente
    @Column(nullable = false) // Especifica que a coluna "id" é obrigatória (não pode ser nula)
    @ToString.Include
    @EqualsAndHashCode.Include
    private long id;

    @Column(nullable = false, unique = true) // Especifica que a coluna "nome" é obrigatória e única (sem repetições)
    @ToString.Include
    private String nome;

}
```

Funcionario

```java
package com.fatecpg.exemplopers.model;

import java.math.BigDecimal;
import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;
import lombok.*;

@Entity
// Especifica o valor que será usado na coluna "tipo_pessoa" para identificar esta subclasse
@DiscriminatorValue("FUNCIONARIO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(onlyExplicitlyIncluded = true)
public class Funcionario extends Pessoa {

    @ToString.Include
    private String cargo;


    @ToString.Include
    private BigDecimal salario;
}
```

Estratégias persistência herança

Single Table (Tabela Única)
Esta é a estratégia **padrão** do JPA. Todas as classes da hierarquia são mapeadas para uma **única tabela** no banco de dados.
•**Como funciona:** O JPA utiliza uma coluna especial chamada **Discriminator Column** (geralmente chamada `DTYPE`) para identificar a qual subclasse aquela linha pertence.
•**Vantagem:** Performance excelente para consultas e polimorfismo, pois não exige *JOINs*.
•**Desvantagem:** Colunas específicas de subclasses devem aceitar valores nulos (`nullable = true`), o que pode comprometer a integridade dos dados no nível do banco.

Joined Table (Tabela por Classe com Join)
Nesta estratégia, cada classe tem sua **própria tabela**, mas as tabelas das subclasses contêm apenas os campos específicos. Elas se conectam à tabela da classe pai através de uma chave estrangeira (que também é a chave primária).
•**Como funciona:** Os dados comuns ficam na tabela pai e os específicos nas tabelas filhas.
•**Vantagem:** Segue as regras de normalização do banco de dados (sem colunas sobrando ou nulas).
•**Desvantagem:** Consultas polimórficas (buscar todos os "Veículos") exigem múltiplos *JOINs*, o que pode degradar a performance em hierarquias complexas.

Estratégias persistência herança

Table Per Class (Tabela por Classe Concreta)
Cada subclasse tem sua própria tabela contendo **todos os campos** (os dela e os herdados da classe pai). A classe pai não possui uma tabela própria se for abstrata.
•**Como funciona:** Não há relacionamento direto entre as tabelas no banco; a herança existe apenas no código Java.
•**Vantagem:** Consultas para uma subclasse específica são muito rápidas.
•**Desvantagem:** Consultas polimórficas são um pesadelo de performance, pois o JPA precisa fazer um `UNION` de todas as tabelas. Além disso, não há suporte para IDs `IDENTITY` (o ID deve ser compartilhado entre as tabelas).

Mapped Superclass (Não é Herança de Entidade)
Anotamos a classe pai com `@MappedSuperclass`. Ela **não é uma entidade** e não possui tabela própria.
•**Como funciona:** Serve apenas para herdar atributos e métodos no Java (como `id`, `dataCriacao`). Cada subclasse gera uma tabela completa com esses campos.
•**Vantagem:** Organização de código repetitivo.
•**Desvantagem:** Você não pode fazer consultas polimórficas (`repo.findAll()` na classe pai não funciona).

Exercício

Criar as funcionalidades para manter cadastro de “Funcionario”.

ATENÇÃO: Não é necessário criar novo serviço e repositor, pois “Funcionario” é um tipo específico de pessoa e os métodos em PessoaService e PessoaRepository e o JPA funcionarão para objetos “Funcionario”.

Programe apenas um novo controller.